DevOps Interview Technical Task

Dear candidate, to showcase technical skills in setting up services, dockerizing, deploying and coding in Golang, please complete the following task and post the result on a public github repo which should at least for the time of the assessment be accessible, and if possible, please deploy to the given server. Details will be provided in a personal Email.

The future work will revolve around handling large amounts of data, having services which can process them well, monitoring those services, and occasionally making adjustments in golang applications. This task gives a chance to showcase all these skills.

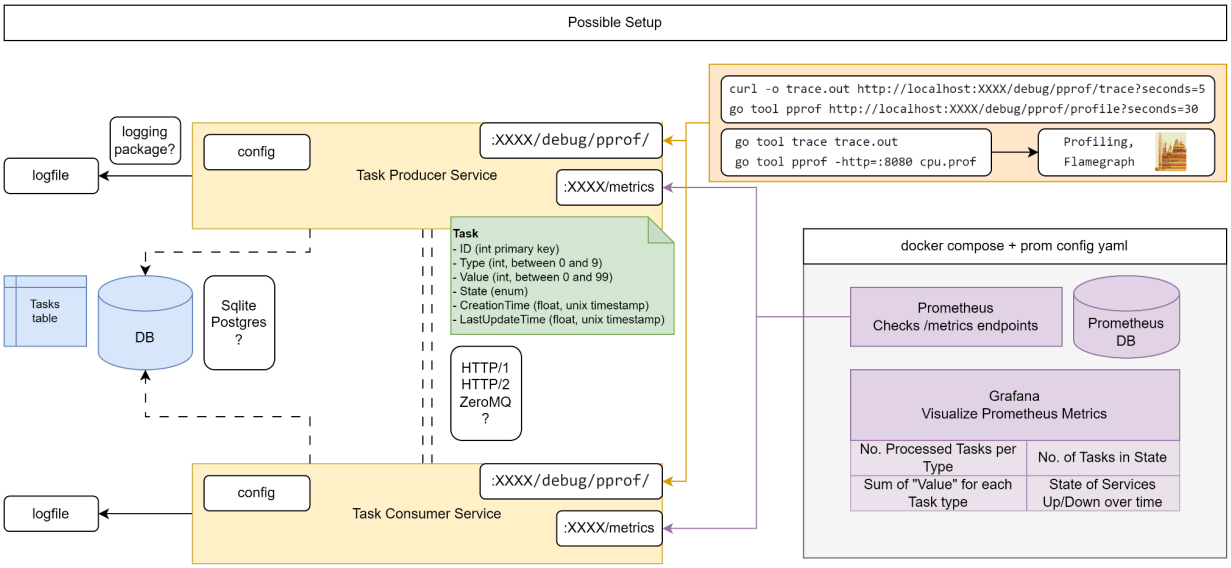
Goal

Goal is to showcase technical understanding of various Golang and IT project related topics which are important when setting up a new project.

Topics to cover

- Understanding the tradeoffs between memory and cpu usage, which can be controlled by GOGC and GOMEMLIMIT parameters, but also throw technical decisions in the code
- Test coverage
- Using the standard library as much as possible
- Using build flag, for example reducing binaries with “`-ldflags="-s -w"`” , building with “pgo” flag or fixing a VERSION variable on build time. Lists available: “go tool link”, “go tool compile”
- Handling configuration
- using “embed”
- Tooling:
    - Using gomigrate to migrate databases
    - Using sqlc to generate golang code for sql queries
- Logging and Monitoring:
    - Choose the right logging library and good log levels in the code
    - Enable monitoring via prometheus
- Profiling
    - How to create a flame graph
    - Use traces, cpu and memory profiles to identify bottlenecks

Description



Services

Create 2 Golang (producer/consumer) Services which communicate with each other via one of the following protocols:

- HTTP/1 REST requests (https://editor.swagger.io/, document with swagger)
- HTTP/2 gRPC requests (https://grpc.io/docs/languages/go/basics/, no documentation needed)
- zeroMQ (https://zeromq.org/languages/go/, only document in words why specific why of communication was chosen, e.g. Req-Reply, Pub-Sub)
- RabbitMQ

The services should be dockerized.

Service 1 - Task Producer

The task producer once started generates request which it sends over the communication channel to the task consumer.
Each task has a task type, which is generated in the producer as a random value (the task type is defined later) which is an integer value between 0 and 9
Each task has a value, which is generated in producer as a random value between 0 and 99
The tasks are persisted in a database to track them by ID and their state. The initial state is “received”
Via prometheus we can track how many tasks have been produced.

Config:

- Prometheus port
- Prometheus endpoint is fixed to “/metrics”
- Configuration based on decision how communication should work between service 1 and service 2
- Max backlog -> this is the highest number of unprocessed messages which the task producer can generate. After this it should stop.
- Logging -> define log level, and choose between console or json structured logs
- Profiling port
- Message production rate -> how many message per second should be produced
- Database connection configuration

Calling the service with the “-version” command line argument should return the version with which the service was built

Service 2 - Task Consumer

The Task Consumer receives the tasks and handles them. Once the task is received, the entry in the DB is set to “processing”
Once the handling is done (an empty function call with sleep, the sleep takes as many milliseconds as the task’s value is set to, so different task will delay the processing for different durations) the database entry for the task’s state is set to “done”
The consumer has a rate limiter, which allows a configured number of incoming tasks within a specified timeperiod.
Track the number of tasks being processed and done in prometheus metrics
Track the number of tasks per task type
Track the total sum of the “value” field for each task type.
For each incoming task, have a final log which describes the tasks content and the calculated total sum for that task’s type.

Config:

- Prometheus port
- Prometheus endpoint is fixed to “/metrics”
- Configuration based on decision how communication should work between service 1 and service 2
- Logging -> define log level, and choose between console or json structured logs
- Profiling port
- Message consumption rate -> rate limit with message consumption
- Database connection configuration

Calling the service with the “-version” command line argument should return the version with which the service was built

Prometheus & Grafana

A simple setup here is enough. A yaml with the prometheus configurations to check the exposed metrics endpoints of both services, and a docker-compose file starting both, prometheus and grafana.

Database

Choose a database technology of your choice for this task. Possible options:

- Sqlite
- Postgres (in docker)

Table:

- Tasks:
    - ID (int primary key)
    - Type (int, between 0 and 9)
    - Value (int, between 0 and 99)
    - State (enum)
    - CreationTime (float, unix timestamp)
    - LastUpdateTime (float, unix timestamp)

Use sqlc to generate the golang files
The persistence package should be reused by both services

Demo

The demo about the project should contain:

- Short explanation on the tech choices for:
    - Communication protocol
    - Database
    - Logging Framework
    - File structure, especially with regards to shared functionality between producer and consumer service
    - Setup of the repo (possible pipelines/releases)
- Starting Service 1 -> Service should run until maximum number of allowed backlog messages is produced. -> visible on grafana and in logs
- Start Service 2 -> Service consumes messages with defined rate limit -> also display on Grafana
- In Grafana there are 4 charts:
    - 1 Chart is setup showing the number of messages in each state “received”, “processing” and “done”
    - 1 Chart is displaying the state of the services, whether they are up and reachable, or down.

    - 1 Chart show the total sum of “value” per task type processed by the consumer service
    - 1 Chart show the total sum processed tasks per task type processed by the consumer service
- Show flame graph of running application
- Show change of performance based on GOGC and GOMEMLIMIT settings
- Show Test Coverage
- Show how the application is built
- Demonstrate that ./producer -version returns the version it was built with
- Demonstrate that ./consumer -version returns the version it was built with
- Perform a migration on the database while the service is running, adding 1 additional column “comment”
- Downgrade the database, removing the additional comment, while the application is running

Allowed Helpers and Materials

- Anything on the internet
- Any contact who can give helpful input
- Any GPT Tool or helpful webpage

We will not check what percentage of the code has been written by you, but whether the code is running well, the technology is well understood and can be extended well.

IP of the Code

Of course we will not use any of the code, and all IP belongs to the candidate. They can use it for any goal, commercial, and/or educational or non-commercial forever.

Evaluation

Not all requirements have to be met and the beauty of grafana dashboards do not matter

- The product will be evaluated based
    - code quality
    - Test coverage
    - Documentation
    - Performance
    - Why/Why not go routines / channels / mutexes were used
- The demo will be evaluated based on:
    - Database migration

        - Profiling of the running system, looking at flamegraphs and generating the memory graph
        - Services should not crash and the entries should be visible in the database
    - Presentation:
        - Explanation of design choices
        - Forward-thinking: Are the choices made such that the solution is scalable? What are bottlenecks?
    - Time Management
        - Many times, trade offs must be made. What trade-offs have been made in building this project
    - Deployment
        - The evaluation will not take into account gitlab/github pipelines, deployment and whether the app itself is dockerized or not

Feedback

The candidate will receive technical feedback on the project and also gets a chance to ask questions on design decisions and technical questions to make it a 2-way exchange.

Guidance

The team is available for questions from the moment the task is set until the task is delivered.

Deadline

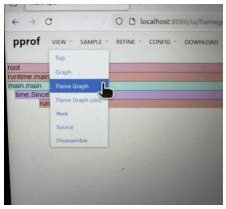
The demo should be given within 15 days from the moment the task is set.
The code must be available and accessible (github/gitlab/gogs/email) at least 24h before the Demo with a minimal description of how to start it.

Helpful materials

https://go.dev/doc/security/fuzz/

https://pkg.go.dev/net/http/pprof

Flame Graph:



https://tip.golang.org/doc/gc-guide

https://go.dev/doc/pgo

https://pkg.go.dev/slices#SortFunc

https://github.com/spf13/viper

https://pkg.go.dev/github.com/golang/mock/mockgen

https://github.com/golang-migrate/migrate

https://github.com/sqlc-dev/sqlc

and of course, golang with grpc
https://grpc.io/docs/languages/go/basics/

and golang with Prometheus
https://prometheus.io/docs/guides/go-application/

Grafana and Prometheus, example stack
https://github.com/vegasbrianc/prometheus/blob/master/docker-compose.yml

Build flags

```
None
go build -ldflags="-help" ./main.go
```

Command showing all ldflags:
go tool link